

The Ouroboros Engine: A Paradigm Shift in Bare-Metal Cryptography

Have you ever wondered how secrets are kept safe on the internet? Or how a tiny microchip can be locked so tightly that even the world's best hackers can't break into it using physics and lasers?

In this deep-dive guide, we are going to explore the magic of **Cryptography** (the science of secret codes) and contrast standard modern crypto against an unbreakable, radically different security framework called the **Ouroboros Engine**.

We will walk through the math step-by-step, entirely by hand, so that you can see exactly how a password like "hello" transforms into a full secret message like "world", and exactly how the math physically destroys the entire payload of data if a hacker types "hxll0".

Part 1: Standard Cryptography (Prime Numbers)

To understand why Ouroboros is so revolutionary, we first have to understand standard cryptography, which relies heavily on **Prime Numbers**.

A prime number is a whole number greater than 1 that can *only* be divided evenly by 1 and itself (e.g., 2, 3, 5, 7, 11). They are the atoms of mathematics. Every whole number is built by multiplying prime numbers together (e.g., $12 = 2 \times 2 \times 3$).

Modern internet security (like the RSA algorithm) relies on a simple mathematical asymmetry: **Multiplication is easy, but factoring is incredibly hard**. If I ask you to multiply 3×11 , you instantly know it's 33. But if I give you a massive 600-digit number and ask you to find the two prime numbers that made it, a supercomputer would take billions of years.

An Example: The RSA “Lock and Trapdoor”

We want to send a secret message—the number **2**—across the internet safely.

Step 1: Generate the Keys We pick two secret prime numbers, $p = 3$ and $q = 11$. We multiply them to create our **Public Lock** (N):

$$N = p \times q = 3 \times 11 = 33$$

Everyone on the internet is allowed to see the number 33.

Step 2: Calculate the “Totient” (ϕ) The Totient (ϕ) counts how many numbers between 1 and 33 share no common factors with 33.

$$\phi(33) = (3 - 1) \times (11 - 1) = 2 \times 10 = 20$$

This number, 20, is our secret backdoor.

Step 3: Choose Encryption and Decryption Keys - We pick an encryption key ($e = 3$) that shares no factors with 20. - We calculate a decryption key (d) where $(d \times e) \div \text{Totient}$ leaves a remainder of 1. Let's test $d = 7$:

$$7 \times 3 = 21$$

$$21 \div 20 = 1 \text{ with a remainder of } 1$$

Our secret decryption key is $d = 7$.

Step 4: Encrypt the Message To encrypt our message ($M = 2$), we raise it to the power of e and take the remainder divided by N .

$$\text{Ciphertext} = 2^3 \pmod{33} = 8 \pmod{33} = 8$$

We send the number **8**. A hacker only sees an 8.

Step 5: Decrypt the Message The receiver uses their secret private key ($d = 7$) to unlock the message:

$$\text{Original Message} = 8^7 \pmod{33} = 2,097,152 \pmod{33}$$

$$2,097,152 \div 33 = 63,550 \text{ with a remainder of } 2$$

The secret message **2** is perfectly recovered.

This works brilliantly for software. But what if a hacker has physical access to the computer chip and can manipulate the electricity itself?

Part 2: Why Modern Crypto Fails at the Hardware Level

In standard software security, the system asks a question:

```
if (password_entered == password_stored) {  
    unlock_data();  
} else {  
    print("Access Denied");  
}
```

This logic relies on a software abstraction. A hacker doesn't need to guess the password. They can use an Electromagnetic Fault Injection (EMFI) tool to shoot a tiny pulse of electricity into the microchip at the exact millisecond it checks the `if` statement. This "glitch" forces the processor to skip the `else` branch, tricking the chip into running `unlock_data()` even if the password was totally wrong!

Hackers also use **Timing Attacks**. If a system takes 5 milliseconds to reject a totally wrong password, but 6 milliseconds to reject a password where the first letter is correct, a hacker can measure the electrical draw and guess the password letter-by-letter based on how long the processor takes to think.

Enter the Ouroboros Engine

The Ouroboros Engine completely abolishes the `if/else` verification step. It is a bare-metal cryptographic framework written in pure AVR Assembly for ATmega328P microchips.

What makes it unlike modern crypto? 1. **Execution IS Decryption:** There is no "authentication" check to bypass. The password *is* the structural architecture of the code. If you enter the wrong password, the software physically decrypts into illegal, scrambled garbage instructions that instantly crash the processor. A glitch attack fails because there is no functional code to execute unless the math resolves perfectly. 2. **The Cryptographic Tar Pit (Immunity to Brute Force):** Ouroboros forces the input password through 24,576 rounds of a Davies-Meyer hash using the Speck-128/256 cipher. Because the chip is intentionally slow (8 MHz), this requires over **3.14 trillion hardware bit-shifts**, acting as a massive 10-second tar pit. A supercomputer cannot speed this up, because the bottleneck is the physical hardware itself. 3. **Absolute Timing Immunity:** The engine uses a branchless (no `if` statements) assembly math trick so that whether the password is 100% right or 100% wrong, the microchip consumes the *exact same number of clock cycles*, leaking zero timing data.

Part 3: The Math of Ouroboros (Modular Arithmetic)

To track how Ouroboros works, we have to understand **Modulo 256**. An 8-bit register (a memory slot on the ATmega chip) can only hold binary numbers from 00000000 to 11111111 (0 to 255). If a calculation goes over 255, it overflows, erasing the big part of the number and keeping only the remainder. Mathematically, this is $\pmod{256}$.

Tracing a Real Example: “hello” unlocks “world”

Let’s see how typing the password “hello” successfully decodes the entire secret word “world”.

Step A: The Passphrase Input The computer converts “hello” into numbers: - h (104), e (101), l (108), l (108), o (111)

It adds them up to establish the base input key:

$$104 + 101 + 108 + 108 + 111 = 532$$

Because 532 overflows the 8-bit register, we take $\pmod{256}$:

$$532 \div 256 = 2 \text{ with a remainder of } 20$$

Our starting key (K_{start}) is **20**.

Step B: The Davies-Meyer “Tar Pit”

[!NOTE] **What real Ouroboros does:** The true Ouroboros hardware uses the full Speck-128/256 cipher (a complex set of bitwise rotations, additions, and XORs) for the encryption step. For this hand-traceable example, we have simplified the block cipher into a basic modulo multiplication (**State $\times$ Key mod 251**), but the Davies-Meyer feed-forward structure (the XOR at the end) remains exactly the same as the real hardware!

The chip traps this key in 24,576 continuous encryption loops. The starting internal state is hardcoded, let’s say $H_0 = 155$. In each round, the cipher multiplies the state by the key, takes $\pmod{251}$, and then uses a bitwise XOR ($\oplus$) to add the original state back in to make it mathematically irreversible.

Round 1: - Encrypt: $155 \times 20 = 3100$ - **Modulo:** $3100 \pmod{251} = 88$ - **XOR Feed-forward:** $88 \oplus 155 = 195$ (New internal state: H_1)

Round 2: - Encrypt: $195 \times 20 = 3900$ - **Modulo:** $3900 \pmod{251} = 135$ - **XOR Feed-forward:** $135 \oplus 195 = 68$ (New internal state: H_2)

After 24,576 rounds, the final stretched key is produced. Let’s say it lands exactly on:

$$K_{\text{final}} = \mathbf{194}$$

Step C: Payload Decryption (Counter Mode)

[!NOTE] **What real Ouroboros does:** In real cryptographic Counter Mode (CTR), the block cipher (Speck) encrypts the counter. For this simplified example, we are generating the modifier using a basic modular multiplication (**Key $\times$ Counter mod 256**), but the underlying concept of XORing a unique, derived modifier against the payload remains identical to the real implementation.

The developer pre-encrypted the target word “world” and saved it in the chip’s raw flash memory. To hide it, they used **Counter Mode (CTR)**. For each letter, they multiply the final key (194) by a counter (1, 2, 3, etc.) $\pmod{256}$ to generate a unique **Modifier**, and XOR it with the target letter.

Let’s trace the full word “world” (ASCII: w=119, o=111, r=114, l=108, d=100):

1. **Letter ‘w’ (119):** Counter = 1. Modifier = $194 \times 1 \pmod{256} = 194$.
 - $119 \oplus 194 = \mathbf{181}$
2. **Letter ‘o’ (111):** Counter = 2. Modifier = $194 \times 2 = 388 \pmod{256} = 132$.
 - $111 \oplus 132 = \mathbf{235}$
3. **Letter ‘r’ (114):** Counter = 3. Modifier = $194 \times 3 = 582 \pmod{256} = 70$.
 - $114 \oplus 70 = \mathbf{52}$
4. **Letter ‘l’ (108):** Counter = 4. Modifier = $194 \times 4 = 776 \pmod{256} = 8$.
 - $108 \oplus 8 = \mathbf{100}$
5. **Letter ‘d’ (100):** Counter = 5. Modifier = $194 \times 5 = 970 \pmod{256} = 202$.
 - $100 \oplus 202 = \mathbf{174}$

The meaningless array [181, 235, 52, 100, 174] is stored permanently in the chip’s flash memory.

When you type “hello”: The tar pit generates the key **194**, pulls the stored array, generates the identical modifiers, and reverses the lock: 1. $181 \oplus 194 = 119$ (**w**) 2. $235 \oplus 132 = 111$ (**o**) 3. $52 \oplus 70 = 114$ (**r**) 4. $100 \oplus 8 = 108$ (**l**) 5. $174 \oplus 202 = 100$ (**d**)

The chip successfully decodes “**world**”!

Part 4: The Branchless Verification Trick (Proving the Math)

Before the chip actually executes the decoded instructions, it must verify the data is authentic. But remember, a standard `if (MAC == valid)` check can be glitched by a hacker!

To defeat this, Ouroboros uses a genius assembly math trick. It compares the decrypted MAC against the expected MAC by subtracting them. The result is called the **Delta**. - If the password was perfect, **Delta** = 0. - If the password was wrong, **Delta** > 0.

The engine places this **Delta** into register **R17**, and runs three pure-math instructions: **NEG**, **SBC**, and **COM**.

Case 1: The Success Path (Password: “hello”)

Because you typed “hello”, the MAC was perfect, so **R17** = 0.

1. **NEG R17 (Two’s Complement Negation):**
 - Negating 0 is still 0.
 - Because the result is exactly zero, the hardware **Carry Flag** is set to 0.
2. **SBC R17, R17 (Subtract with Carry):**
 - This instruction subtracts a register from itself, minus the Carry flag ($R17 - R17 - \text{Carry}$).
 - Math: $0 - 0 - 0 = 0x00$ (All zeros).
3. **COM R17 (Bitwise NOT):**
 - This inverts every bit. $0x00$ (00000000) becomes $0xFF$ (11111111).

Our mathematical mask is **0xFF**. The engine takes the fully decrypted payload (“world”) and bitwise ANDs every single letter against the mask ($0xFF$ preserves data): - 119 (‘w’) & $0xFF$ = **119** - 111 (‘o’) & $0xFF$ = **111** - 114 (‘r’) & $0xFF$ = **114** - 108 (‘l’) & $0xFF$ = **108** - 100 (‘d’) & $0xFF$ = **100**

The entire payload **world** is perfectly intact and executes successfully!

Case 2: The Failure Path (Password: “hxllo”)

What if a hacker typos “hello” as “hxllo”? The letter ‘x’ (120) changes the base key. The Avalanche Effect violently scrambles the tar pit, producing a completely wrong final key. Let’s say the new key is **43**.

When the chip tries to decrypt the flash memory [181, 235, 52, 100, 174], it generates completely wrong modifiers: - Counter=1: $43 \times 1 = 43$. - $181 \oplus 43 = \mathbf{158}$ - Counter=2: $43 \times 2 = 86$. - $235 \oplus 86 = \mathbf{189}$ - Counter=3: $43 \times 3 = 129$. - $52 \oplus 129 = \mathbf{181}$ - Counter=4: $43 \times 4 = 172$. - $100 \oplus 172 = \mathbf{200}$ - Counter=5: $43 \times 5 = 215$. - $174 \oplus 215 = \mathbf{121}$

It decrypted to the garbage array [158, 189, 181, 200, 121]. But it gets even more brilliant.

The engine calculates the **Delta** for the MAC. Because the keys didn’t match, the **Delta** is a non-zero number. Let’s say **R17** = 45.

1. **NEG R17 (Two’s Complement Negation):**
 - Negating 45 results in -45.
 - Because the starting number (45) was *not* zero, the hardware **Carry Flag** is forcibly set to 1.
2. **SBC R17, R17 (Subtract with Carry):**
 - Equation: $R17 - R17 - \text{Carry}$
 - Math: Because NEG changed R17 to -45, the ALU executes $-45 - (-45) - 1 = -1$.
 - In 8-bit binary, -1 wraps around to $0xFF$ (11111111).
3. **COM R17 (Bitwise NOT):**
 - Inverting $0xFF$ (11111111) becomes $0x00$ (00000000).

Our mathematical mask is now **0x00**. The engine takes the garbage payload and bitwise ANDs every single byte against the mask ($0x00$ destroys data): - 158 & $0x00 = \mathbf{0}$ - 189 & $0x00 = \mathbf{0}$ - 181 & $0x00 = \mathbf{0}$ - 200 & $0x00 = \mathbf{0}$ - 121 & $0x00 = \mathbf{0}$

The **entire payload** is instantly and physically destroyed, overwritten with [0, 0, 0, 0, 0]. In AVR assembly, $0x0000$ is the NOP (No Operation) instruction. Because AVR reads 16-bit instructions, this 5-byte

payload translates to two harmless NOP instructions, with the 5th byte helplessly combining with whatever random memory comes next. Instead of executing garbage or allowing a hacker to glitch past a check, the chip executes blank space and derails safely.

Case 3: The Firmware Patch Attack (Forcing a Success)

Hackers have one last hardware trick. If they manage to extract the chip's ROM and reprogram it (patching the firmware), they might try to simply overwrite the NEG, SBC, COM instructions.

In a standard system, a hacker can patch `if (password == valid)` to `if (true)`. They instantly win, and the locked data is handed to them.

What happens if they patch Ouroboros to force the mask to **0xFF**, even when they type the wrong password ("hxlllo")?

Let's assume the hacker injects malicious assembly instructions to force the mask to always be 0xFF (data preservation):
- 158 & 0xFF = **158** - 189 & 0xFF = **189** - 181 & 0xFF = **181** - 200 & 0xFF = **200** - 121 & 0xFF = **121**

The hacker successfully bypasses the mathematical mask. **But they still lose.** Because the password was wrong, the CTR decryption step mathematically failed to unravel the lock. The payload that the processor is now forced to execute is [158, 189, 181, 200, 121].

Let's be **100% explicit** about what the CPU actually executes here. The AVR processor reads instructions in 16-bit pairs (Little-Endian).

The garbage array is [158, 189, 181, 200, 121]. In Hexadecimal, this is [0x9E, 0xBD, 0xB5, 0xC8, 0x79].

The CPU reads the first two bytes (0x9E and 0xBD) and constructs the 16-bit instruction **0xBD9E**. In AVR Assembly, the binary opcode 1011 1101 1001 1110 translates directly to:

```
OUT 0x2E, R25    ; Write the contents of Register 25 into I/O address 0x2E
```

On the ATmega328P, I/O address 0x2E is the **SPCR** (SPI Control Register), not the Stack Pointer! While the author claimed this corrupts the stack, it actually corrupts the hardware SPI interface. However, the fatal crash still occurs on the very next instruction!

The CPU then blindly reads the next two bytes (0xB5 and 0xC8) to construct the instruction **0xC8B5**. In AVR Assembly, the binary opcode 1100 1000 1011 0101 translates to:

```
RJMP -1867       ; Relative Jump backwards by 1,867 words
```

With the hardware's SPI interface now completely corrupted, the CPU blindly executes the next opcode and wildly jumps 1,867 instructions backwards in memory into the absolute abyss. The execution derails completely, causing an inescapable, fatal hardware crash.

Because the password was wrong, the data structurally transformed into an immediate, unavoidable self-destruct sequence.

Why is this Unbreakable?

A hacker cannot glitch past the `if` statement, because **there is no if statement**. The math itself destroys the payload if the key is even slightly off. Furthermore, a hacker cannot patch the firmware to bypass the security, because the security isn't just a lock on a door—it is the structural architecture of the data itself. The password mathematically reconstructs reality. Without the exact correct password, the resulting data is just toxic noise that obliterates the CPU.

The execution path is identical down to the exact transistor clock cycle, leaking zero timing data, and ensuring that Ouroboros remains an impenetrable fortress of bare-metal security.

Part 5: Evolution to RISC-V (The Ouroboros Badge)

The tutorial above explores the original 8-bit AVR implementation. However, modern iterations of this framework have evolved to support 32-bit architectures, most notably in the Ouroboros Badge (Tindie).

The Ouroboros Badge utilizes a **RISC-V RV32EC** microcontroller (specifically the CH32V003). Moving from 8-bit AVR to 32-bit RISC-V completely changes the hardware landscape.

The Missing Carry Flag

In the AVR example, our branchless verification trick relied on the `NEG`, `SBC`, and `COM` instructions interacting with the CPU's hardware **Carry Flag**.

RISC-V is a modern, streamlined architecture that deliberately **does not have a Carry Flag** or a Status Register. This makes the AVR assembly trick impossible. Instead, RISC-V relies on instructions that write comparison results directly into general-purpose registers.

The 32-bit RISC-V Branchless Trick

To achieve the exact same zero-timing branchless destruction on RISC-V, Ouroboros uses the `SLTU` (Set Less Than Unsigned) instruction.

Let's assume our MAC comparison `Delta` is stored in the 32-bit register `x10`. - If the password is perfect, `x10 = 0`. - If the password is wrong, `x10 > 0`.

The RISC-V branchless mask is achieved in just two instructions:

```
; x10 contains the Delta (0 = Success, >0 = Failure)

SLTU x11, x0, x10    ; Set Less Than Unsigned
                     ; If 0 < x10, set x11 to 1 (Failure)
                     ; If 0 is NOT < x10 (meaning x10 is 0), set x11 to 0 (Success)

ADDI x11, x11, -1     ; Add Immediate -1
                     ; If x11 was 0 (Success): 0 - 1 = -1 (0xFFFFFFFF)
                     ; If x11 was 1 (Failure): 1 - 1 = 0 (0x00000000)
```

Just like the AVR version, this generates an inescapable mathematical mask (0xFFFFFFFF for success, 0x00000000 for failure). The engine takes the decrypted 32-bit payload and bitwise ANDs it against `x11`: - **Success:** `Payload & 0xFFFFFFFF = Payload` (Executes perfectly). - **Failure:** `Payload & 0x00000000 = 0x00000000` (Destroys the payload).

In RISC-V architecture, the opcode 0x00000000 is defined as an **Illegal Instruction** (or `C.ILLEGAL` in RV32C compressed instructions).

So if a hacker enters the wrong password, the payload is physically zeroed out, and the moment the processor tries to read it, it instantly triggers an Illegal Instruction exception and crashes—exactly as it did on the 8-bit AVR.

The architecture changed, but the inescapable tar pit remains.

Part 6: A Simple End-to-End Full Explanation and Example

Here is the complete, hand-calculated trace of the Ouroboros Engine using exactly three Davies-Meyer rounds.

To make this work, the original target payload in the flash memory must be pre-encrypted using the exact final key that a 3-round tar pit will generate.

Step A: The Passphrase Input

We start by deriving the base key from the input “hello”.

- **ASCII Conversion:** h (104), e (101), l (108), l (108), o (111)
- **Summation:**

$$104 + 101 + 108 + 108 + 111 = 532$$

- **Modulo 256 Overflow:**

$$532 \pmod{256} = 20$$

Our starting key (K_{start}) is **20**.

Step B: The 3-Round Davies-Meyer “Tar Pit”

[!NOTE] **What real Ouroboros does:** Just like in the earlier example, we are using modulo multiplication as a stand-in for the incredibly complex Speck cipher, so you can trace the math by hand. The Davies-Meyer feed-forward structure (**XOR Result**) is completely authentic to the hardware!

The chip begins at a hardcoded internal state of $H_0 = 155$. In each round, it multiplies the state by the key, takes $\pmod{251}$, and bitwise XORs ($\oplus$) the previous state back in.

Round 1

- **Encrypt:** $155 \times 20 = 3100$
- **Modulo 251:** $251 \times 12 = 3012$.

$$3100 - 3012 = 88$$

- **XOR Feed-forward:** $88 \oplus 155$
- 88 in binary: 0101 1000
- 155 in binary: 1001 1011
- XOR Result: 1100 0011 (which is $128 + 64 + 2 + 1$)
- **New State (H_1): 195**

Round 2

- **Encrypt:** $195 \times 20 = 3900$
- **Modulo 251:** $251 \times 15 = 3765$.

$$3900 - 3765 = 135$$

- **XOR Feed-forward:** $135 \oplus 195$

- 135 in binary: 1000 0111
- 195 in binary: 1100 0011
- XOR Result: 0100 0100 (which is $64 + 4$)
- **New State (H_2): 68**

Round 3 (The Final Round)

- **Encrypt:** $68 \times 20 = 1360$
- **Modulo 251:** $251 \times 5 = 1255$.

$$1360 - 1255 = 105$$

- **XOR Feed-forward:** $105 \oplus 68$
 - 105 in binary: 0110 1001
 - 68 in binary: 0100 0100
 - XOR Result: 0010 1101 (which is $32 + 8 + 4 + 1$)
 - **Final Key (K_{final}): 45**
-

Step C: Pre-Encryption & Payload Decryption (Counter Mode)

[!NOTE] **What real Ouroboros does:** We are again using our simplified $\text{Key} \times \text{Counter} \bmod 256$ math to generate modifiers, rather than running the counter through the full Speck cipher.

Because our 3-round tar pit generates **45** as the final key, the developer must pre-encrypt the word “world” into flash memory using this specific key.

Here is the exact math the developer used to store the array, which the chip will now reverse:

1. Letter ‘w’ (119)

- Counter = 1. Modifier = $45 \times 1 = 45$.
- Encryption: $119 \oplus 45$
- 119: 0111 0111
- 45: 0010 1101
- Result: $0101\ 1010 = \mathbf{90}$

2. Letter ‘o’ (111)

- Counter = 2. Modifier = $45 \times 2 = 90$.
- Encryption: $111 \oplus 90$
- 111: 0110 1111
- 90: 0101 1010
- Result: $0011\ 0101 = \mathbf{53}$

3. Letter ‘r’ (114)

- Counter = 3. Modifier = $45 \times 3 = 135$.
- Encryption: $114 \oplus 135$
- 114: 0111 0010
- 135: 1000 0111
- Result: $1111\ 0101 = \mathbf{245}$

4. Letter ‘l’ (108)

- Counter = 4. Modifier = $45 \times 4 = 180$.

- Encryption: $108 \oplus 180$
- 108: 0110 1100
- 180: 1011 0100
- Result: 1101 1000 = **216**

5. Letter ‘d’ (100)

- Counter = 5. Modifier = $45 \times 5 = 225$.
- Encryption: $100 \oplus 225$
- 100: 0110 0100
- 225: 1110 0001
- Result: 1000 0101 = **133**

The encrypted payload stored permanently in the chip’s ROM is [90, 53, 245, 216, 133].

The Decryption Pass When the user types “hello”, the tar pit correctly yields $K_{\text{final}} = 45$. The chip reverses the XOR operations perfectly against the stored ROM array:

1. $90 \oplus 45 = \mathbf{119}$ (‘w’)
2. $53 \oplus 90 = \mathbf{111}$ (‘o’)
3. $245 \oplus 135 = \mathbf{114}$ (‘r’)
4. $216 \oplus 180 = \mathbf{108}$ (‘l’)
5. $133 \oplus 225 = \mathbf{100}$ (‘d’)

Step D: The Branchless Verification Trick

Now the chip must verify if this decrypted payload is safe to execute, without using an `if` statement.

Because the calculated final key (45) matched the expected final key perfectly, the MAC (Message Authentication Code) resolves with zero variance. The engine calculates the **Delta** and stores it in register R17:

$$\text{Delta (R17)} = 0$$

The CPU executes the three constant-time math instructions:

1. **NEG R17**: Negating 0 leaves 0. The Carry Flag is set to 0.
2. **SBC R17, R17**: $0 - 0 - 0 = \mathbf{0x00}$ (0000 0000).
3. **COM R17**: Bitwise inversion flips 0000 0000 to **0xFF** (1111 1111).

The mathematical mask is locked in as **0xFF**. The engine takes the decrypted payload and applies a bitwise AND (&) against this mask:

- $119 \& 0xFF = \mathbf{119}$
- $111 \& 0xFF = \mathbf{111}$
- $114 \& 0xFF = \mathbf{114}$
- $108 \& 0xFF = \mathbf{108}$
- $100 \& 0xFF = \mathbf{100}$

The payload is mathematically preserved. The system accepts the data as valid, authentic instructions, and the execution of “world” proceeds flawlessly.